AI-Assisted Development

Blue Harvest · Mortgage Pricing Exceptions

This document describes how AI-assisted development supported the assignment and how its output was checked. It distinguishes recorded local evidence from review and production work that remains outstanding.

1. Tools and Working Approach

1.1 AI assistant used

OpenAI Codex was used as the coding assistant in the developer's workspace. It inspected repository files, proposed and applied changes, ran commands and interpreted build and test results. No autonomous sub-agents were used in the recorded work. This document does not attribute work to other AI tools without evidence.

1.2 Developer direction and accountability

The developer directed the work through incremental requests covering implementation, business naming, tests, Docker execution and submission documents. Codex performed those tasks and reported results. Business policies were documented as assumptions where the assignment left details open. The developer remains responsible for accepting the implementation, confirming policies and completing the final submission review.

2. Contributions to the Assignment

2.1 Design and business rules

Codex helped interpret the assignment and explain the conflict between allowing changed terms and preserving exact approvals. The implementation uses new immutable revisions: revised terms return to Pending and require a fresh decision. It also helped document external mortgage references, reviewer assignment, retry handling, concurrency limits and the distinction between exercise policies and production requirements.

2.2 Application code

The assistant drafted and revised API controllers, DTOs, services, EF Core persistence, development identity simulation and error handling. It supported PostgreSQL and Docker setup, the React frontend and user selection. Later tasks renamed public actions, service methods, private helpers and local variables to reflect the business domain while preserving routes and behavior.

2.3 Tests and documentation

Codex helped create and maintain service, HTTP integration, PostgreSQL and browser tests, plus live workflow and persistence scripts. It generated or refined the README, assumptions, diagrams, Word/PDF guides and presentation. Python document libraries and Mermaid rendering tools produced document files and diagram images; they were supporting automation, not additional AI decision agents.

3. Review, Validation and Remaining Limits

3.1 Checks applied to generated output

- Requirement alignment: repository code and documents were compared with the assignment brief. Assumptions and deliberate exclusions were separated from client requirements.

- Source review: changes and references were inspected for unintended edits. Naming refactors were checked against interfaces and callers, with particular attention to DTO fields, persisted retry input, authorization and existing routes.

- Compiler and lint feedback: backend and frontend builds and frontend lint checks were run. Failed checks were used to correct generated changes. For example, compiler errors caught unintended type/property substitutions during variable renaming before the corrected version passed validation.

- Behavioral tests: automated checks covered creation, approval, decline, revisions, current-discount lookup, history, idempotent retries, conflicting input, stale updates, roles and assigned-reviewer restrictions.

- Deployment checks: the Docker HTTP smoke test exercised the actual PostgreSQL-backed application. Image verification identified a stale frontend container, which was recreated and checked again. Persistence checks verified retained records across container recreation.

- Document checks: file links, command paths, method references and generated document packages were checked. Diagram images and presentation layouts were also inspected during the documentation work.

3.2 Recorded execution evidence

Earlier checks in this workspace completed successfully: backend build with zero warnings and errors; 184 backend tests (56 unit, 118 HTTP integration and 10 PostgreSQL); frontend build and lint; 20 isolated browser tests; one multi-user browser workflow against Docker; API smoke tests; and persistence across Compose container recreation. These describe the source state tested at that time, not a new test run for this document or evidence that remote CI has executed.

3.3 Limits and final engineering review

Generated code and generated tests can share the same mistaken assumption. Passing local checks therefore does not prove complete requirement coverage, production security, multi-instance correctness or recovery readiness. A clean-machine run and independent final code review should be completed against the intended submission commit. No claim is made here that those independent reviews have already occurred.

The application does not use AI to approve mortgage discounts. Human reviewers, represented by development identities in the demo, record approval or decline decisions. Production identity, real mortgage-system validation, stronger distributed concurrency controls and operational readiness remain separate work.

The developer reported 25 hours of work over five days at five hours per day. This is the developer-provided total, not a measurement of Codex execution time.

3.4 Supporting Record

Source record: Documentation/Architecture/AiDevelopmentLog.md and the development conversation. The older log contains historical test totals and an earlier missing time estimate; this document uses later observed test runs and the developer's supplied time. Deliverables/README.md, Engineering_Decisions_and_Operational_Guidance.docx and Production_Readiness.docx provide the current handoff context. No application code was modified or tests executed to create this document.